arXiv:2611.12760v1 [cs.CR] 9 Nov 2026

DEEPCHRONOS: TEMPORAL NEURO-SYMBOLIC REASONING FOR MULTI-STEP ATTACK DETECTION

A PREPRINT

Sofia Burkow[1] **Andrea Agiollo**[1] **Enkeleda Bardhi**[1] **Fernando Kuipers**[1]
Kaitai Liang[2] **Georgios Smaragdakis**[1]

[1]Delft University of Technology [2]University of Turku

sofiaburkow@proton.me, kaitai.liang@utu.fi,
{A.Agiollo-1, E.Bardhi-1, F.A.Kuipers, G.Smaragdakis}@tudelft.nl

ABSTRACT

Modern cyber threats commonly unfold as multi-step attacks comprising temporally and causally related phases. Current defenses commonly follow a *detect first and correlate later* paradigm. Intrusion detection systems classify events independently, while downstream systems reconstruct attack progression from the resulting alerts or audit records. In this context, learning-based detectors learn attack-step dependencies implicitly as statistical regularities in the data and cannot guarantee that their predictions follow the real attack progression. Conversely, correlation systems represent attack relationships explicitly, but apply this knowledge only after the underlying attack observations have been detected. We present DEEPCHRONOS, a neuro-symbolic network intrusion detection system that shifts the paradigm from *detect first and correlate later* to *detection through logical progression*. DEEPCHRONOS combines learned flow-level evidence with explicit attack-phase rules, temporal prerequisites, and previously inferred phases, allowing the evolving attack history to inform and constrain each detection decision. Evaluations on DARPA 2000 and AIT-LDS V2 show that DEEPCHRONOS improves macro F1 over purely neural baselines by up to 11.3%, eliminates causal violations under strict temporal constraints, and maintains millisecond-scale inference latency. These results demonstrate that integrating explicit attack progression knowledge into detection produces more accurate and temporally coherent multi-step attack predictions.

Keywords Multi-step Attacks · Neuro-symbolic AI · Probabilistic logic programming · Explainable Security

1 Introduction

Modern cyber-attacks rarely consist of a single malicious event. Adversaries pursue their objectives through interdependent steps, such as reconnaissance, initial access, privilege escalation, lateral movement, and more as reflected in the Cyber Kill Chain and MITRE ATT&CK [17, 35]. Detecting these multi-step attacks (MSAs) is challenging because their actions may be distributed across hosts, separated in time, and insufficiently suspicious when considered in isolation [41, 13]. Together, however, they can cause severe harm. For example, in the SolarWinds incident, the adversary combined supply-chain access with credential and identity abuse, persistence, lateral movement, collection, and exfiltration [37]. More recently, in the 2024 Change Healthcare intrusion, attackers used compromised credentials to access a remote portal without multifactor authentication, moved laterally, exfiltrated data, and deployed ransomware nine days later [58].

Both incidents show how the final impact was preceded by a progression of consequential activities, illustrating why detecting only the terminal event is insufficient for effective defense. Moreover, detecting an MSA early requires recognizing not only its constituent steps, but also the temporal and causal relations connecting them. For example, lateral movement generally presupposes an established foothold or valid credentials, while exfiltration presupposes access to relevant data. Although attackers may skip, repeat, or interleave activities, attack steps remain subject to prerequisite and partial-order relationships [17, 41]. These relations allow weak observations to reveal a developing

intrusion. Conversely, predictions such as exfiltration before compromise may be locally plausible yet inconsistent with the evolving attack context. An effective detector should therefore use previously observed steps to guide each new prediction and identify the attack before its objective is reached.

Most existing defenses against MSAs follow a *detect first, correlate later* paradigm. In the first stage, an IDS examines network or system observations and produces independent alerts. Signature- and rule-based IDSs detect known behaviors using expert-defined patterns, thus typically recognize atomic actions rather than attack progressions [53, 55]. Meanwhile, machine learning (ML)-based IDSs provide greater flexibility by learning malicious or anomalous behavior from data [33, 56, 1, 4] and aiming to learn attack-step dependencies implicitly as statistical regularities—e.g., using recurrent architectures such as LSTMs [15] and GRUs [6]. However, because these dependencies are only learned as statistical correlations, the model may predict phases whose prerequisites have not been observed and produce invalid attack progression predictions.

In the second stage, alert-correlation systems attempt to recover the attack progression by grouping alerts and reconstructing higher-level attack scenarios [41, 13, 57]. These systems make attack-step relationships explicit and can support both online analysis and post-incident investigation. Nevertheless, their reasoning remains downstream of detection. The alerts have already been generated before the attack context is considered. Consequently, attack progression knowledge cannot influence how each event is classified and false alerts may produce spurious attack paths. The conventional pipeline therefore exposes a fundamental separation: detection systems process individual events without explicit attack steps knowledge, while correlation systems reason about attack steps only after the detection decisions have been made.

We propose to overcome this separation by explicitly enforcing the logical attack progression knowledge within the detection system—i.e., *detection through logical progression*. Rather than learning attack-step relationships only implicitly or applying them only during downstream correlation, progression knowledge should directly inform the detection of each new observation. To this end, we present DEEPCHRONOS, the first neuro-symbolic IDS that embeds explicit MSA progression logic directly into attack detection. Neuro-symbolic systems combine the representation-learning capabilities of neural networks with explicit reasoning of symbolic AI [10, 30, 7, 51]. Similarly, DEEPCHRONOS uses a neural component to identify malicious behavior from noisy network-flow data and a symbolic reasoning engine to encode attack-phase rules and temporal prerequisites of MSAs. For each flow, it reasons jointly over learned evidence and the previously observed history of the relevant attack context, allowing earlier steps to inform and constrain subsequent predictions. A lightweight memory maintains separate histories for active contexts, supporting attacks that unfold over extended periods. Unlike conventional IDSs and post-hoc correlation systems, DEEPCHRONOS integrates uncertain neural evidence and explicit MSA logic within probabilistic attack-phase inference. Thus, DEEPCHRONOS raises an alarm when the accumulated context satisfies a configurable multi-step attack criterion, enabling weak early evidence to contribute to detection while limiting alarms from isolated predictions.

This paper makes the following contributions:

- We formulate MSA detection as context-dependent neuro-symbolic inference governed by explicit temporal and attack-phase knowledge.
- We design DEEPCHRONOS, which integrates neural flow classification, probabilistic logical reasoning, and context-specific attack memory in an end-to-end pipeline.
- We introduce metrics for causal and regression violations and evaluate DEEPCHRONOS on DARPA 2000 [25] and AIT-LDS V2 [23, 22] against neural, symbolic, and anomaly-detection baselines.
- We analyze DEEPCHRONOS data efficiency, cross-domain generalization, robustness to missed steps, and runtime overhead.

The remainder of this paper is organized as follows. Sections 2 and 3 present the background and threat model, respectively. Section 4 describes DEEPCHRONOS architecture, while Section 5 details its implementation and Section 6 elaborates on its evaluation. Lastly, Section 7 discuss limitations and Section 8 the related work, while Section 9 concludes the paper.

2 Background

2.1 Multi-Step Attacks

A multi-step attack (MSA) comprises related actions performed toward a common adversarial objective [40]. Attack frameworks describe these actions as phases such as reconnaissance, initial access, persistence, lateral movement, collection, and exfiltration [35, 17]. We denote the modeled phases by $\mathcal{P} = \{p_1, \ldots, p_M\}$ and an attack progression

by a sequence over $\mathcal{P}$. A prerequisite relation $p_i \prec p_j$ indicates that p_i must be observed before p_j to be considered plausible. Depending on the attacker, phases may form a strict sequence or there may exist omissions and phase repetitions, as real attackers may skip, repeat, or interleave activities. MSA detection therefore requires both classifying individual phases and determining whether their accumulation indicates an ongoing attack. Predicting a phase before a required predecessor constitutes a *causal violation*, whereas predicting an incompatible earlier phase constitutes a *regression violation*.

2.2 Network Intrusion Detection Systems

A network intrusion detection system (NIDS) monitors traffic for malicious activity or policy violations [53]. Signature-based NIDSs match expert-defined attack descriptions, while data-driven NIDSs learn malicious or anomalous traffic patterns [55]. NIDSs may analyze packets or aggregate them into flows, typically identified by endpoint addresses, ports, and transport protocol. Flow features summarize properties such as duration, traffic volume, directionality, and inter-arrival times, and remain available when payloads are encrypted. A learned NIDS commonly estimates $p_\theta(Y = y \mid x)$ for a traffic observation x and an attack class prediction y, while MSA detection additionally conditions phase inference on previous phases and attack-progression knowledge.

2.3 Neuro-Symbolic Reasoning

Neuro-symbolic AI combines neural representation learning with explicit knowledge representation and symbolic reasoning [10, 30, 7, 51]. Neural models extract uncertain patterns from data, while symbolic models express domain knowledge through facts and rules.

DeepProbLog. DeepProbLog [27, 28] extends the probabilistic logic language ProbLog with neural predicates [8]. A logic program represents knowledge through *facts*, *predicates*, and *rules*. A fact states that a relation holds, while a predicate represents a general relation. Rules derive new facts from existing knowledge. For example, a rule may identify a scanning phase from the protocol and number of contacted ports. ProbLog assigns probabilities to uncertain facts, defining possible worlds in which different facts hold. A query's probability is then computed as the total probability of the worlds in which it can be proven. DeepProbLog preserves this semantics while allowing selected probabilities to be produced by neural networks. A *neural predicate* maps an input x, such as a network flow, to a probability distribution over symbolic values, such as benign and malicious. These values can participate in logical rules together with deterministic and probabilistic knowledge. To answer a query, DeepProbLog evaluates the required neural predicates and performs probabilistic inference over the logical proofs supporting the query. Under end-to-end training, query-level gradients propagate through inference to the neural parameters, allowing learning and reasoning to be optimized jointly.

3 Threat Model

3.1 Attacker Model

We consider adversaries whose actions form a MSA progression consistent with established lifecycle models, such as the Cyber Kill Chain [26]. Although the concrete techniques used at each stage may vary, we assume that the attack retains a staged structure comprising reconnaissance, initial compromise, and post-compromise activities. Similar assumptions underlie systems such as HOLMES, which models coordinated activities including reconnaissance, command and control, privilege escalation, lateral movement, and exfiltration [32]. Within the scope of this work, we further assume that the modeled attack phases produce observable network-flow characteristics. This assumption limits the system to the network-visible portion of an attack. The host activities that do not generate distinctive network traffic are outside the current detection scope. Finally, we allow related attack flows to be separated by extended intervals, ranging from hours to days. The system must therefore retain historical information beyond the neural input window to associate temporally distant observations. This is consistent with prior work showing that attack behavior may span multiple flows and extended time periods [48], as well as with provenance-based systems designed to preserve historical context for long-running attacks [14].

3.2 Evasion Assumptions

We consider non-adaptive adversaries whose actions preserve the network and temporal characteristics of the modeled multi-step attacks. Specifically, we assume that attackers do not possess detailed knowledge of DEEPCHRONOS or deliberately alter their behavior to evade its neural classification and symbolic correlation mechanisms. Potential adaptations include delaying, reordering, or omitting attack phases, etc. Such evasion strategies are feasible, as prior work

has demonstrated practical problem-space attacks against learned security models and behavioral mimicry against provenance-based intrusion detectors [47, 12]. However, evaluating these strategies requires an adversarial methodology that preserves the attack objective and relevant network constraints. We therefore restrict the current evaluation to non-adaptive executions of the modeled attack scenarios and leave robustness against adaptive adversaries outside the scope of this work.

3.3 Missed Attacks and False Alarms

The objective of DEEPCHRONOS is to detect multi-step attacks before their final objective is reached. Early-stage activities, such as reconnaissance, are often ambiguous in isolation, but missing them may leave the attack history incomplete and hinder subsequent progression reasoning. Later phases, such as exploitation and post-compromise activity, provide stronger evidence and therefore require higher detection reliability. To balance missed detections and false alarms, individual phase predictions are stored as evidence but do not directly trigger an alarm. An alarm is raised only when multiple temporally consistent phases satisfy the predefined multi-step attack criteria. This preserves weak early-stage evidence while reducing the likelihood that an isolated false prediction results in an alarm, following a distinction between individual suspicious events and coordinated attack campaigns also found in HOLMES and ATLAS [32, 2].

3.4 Target Environment

DEEPCHRONOS targets continuous multi-step attack detection in medium- to large-scale networks, where low inference latency, bounded state, and sustained flow processing are required. Because excessive false alarms can overwhelm security operators, the system is designed to raise an alarm only after multiple temporally consistent attack phases satisfy the predefined multi-step attack criteria. The system assumes access to live or captured network traffic processed by a network monitor capable of extracting structured flow records. To remain applicable when application traffic is encrypted, DEEPCHRONOS relies exclusively on packet-header and connection-level metadata, including protocol, endpoints, ports, direction, duration, and traffic-volume statistics. Payload inspection and host-only activity that produces no observable network evidence are outside the current scope.

4 DEEPCHRONOS

4.1 Overview

DEEPCHRONOS introduces a neuro-symbolic MSA detection system built using DeepProbLog [27, 29], combining neural attack-phase classification with probabilistic symbolic reasoning. Prior systems commonly reconstruct attack progression by correlating IDS alerts or causally related audit events into attack graphs and attack stories [38, 32, 2]. In contrast, DEEPCHRONOS integrates explicit attack-progression knowledge directly into flow-level inference, allowing previously observed phases to constrain subsequent attack-phase predictions.

Figure 1 provides an overview of the proposed multi-step attack detection system. During data preparation, incoming network traffic captures are transformed into network flows, statistical features are computed, and DeepProbLog queries are constructed for neuro-symbolic attack-phase inference. During inference, both the neural and symbolic components reason about the most likely attack phase, after which a joint decision is made. If an attack phase is detected, it is stored in a lightweight database that maintains the attack history. This attack history is then used to provide contextual information for subsequent predictions. Finally, a multi-step attack alarm is raised if the accumulated attack history satisfies predefined MSA criteria.

Subsequently, we explain in detail each DEEPCHRONOS component, including: *(i)* the data preparation phase that transforms incoming raw traffic captures into a neuro-symbolic-friendly format that is used for inference (Section 4.2); *(ii)* the neuro-symbolic attack-phase inference that combines a neural component with temporal, symbolic reasoning (Section 4.3), and *(iii)* the multi-step attack detection decision (Section 4.4).

4.2 Data Preparation

The data preparation component is responsible for transforming incoming network traffic captures into a format that can be used for neuro-symbolic inference.

Network Flows. The first stage of the pipeline transforms raw network traffic captures into network flows. A network flow is defined as a grouping of packets sharing the same 5-tuple (source IP, destination IP, source port, destination port, and protocol) within a given time interval [20].

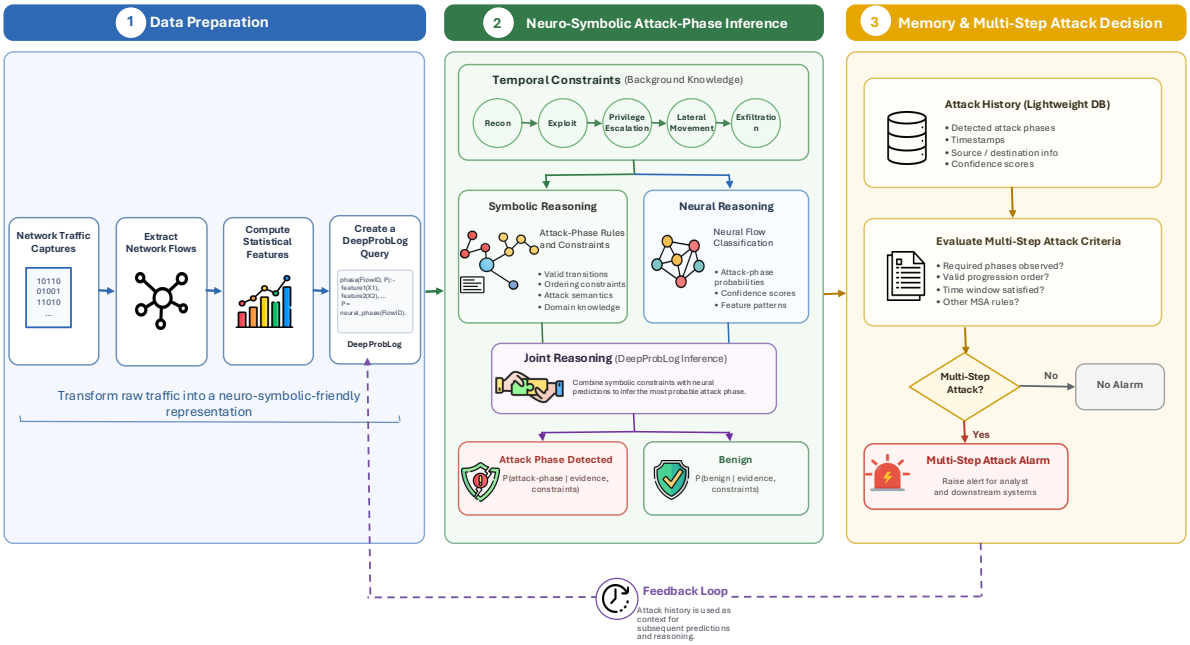


Figure 1: DEEPCHRONOS architecture. Network traffic is transformed into flow-level features and DeepProbLog queries, jointly evaluated through neural classification and symbolic temporal reasoning. Detected attack phases are stored as contextual history and evaluated against multi-step attack criteria to raise an alarm.

This abstraction provides a condensed view of network activity, capturing communication patterns, volume statistics, and relationships between endpoints. In addition, flows are inherently sequential, allowing observations to be analyzed in the context of preceding and subsequent activity. This is particularly useful for detecting attack behaviors that only become apparent when considered across multiple events, making flow-based representations well suited for ML approaches that rely on temporal or relational structure. Lastly, we design DEEPCHRONOS to operate on encrypted traffic and therefore exclusively extracting flow features from packet header fields.

Statistical Features. Certain attack behaviors cannot be fully captured using individual network flows, as they emerge from patterns across multiple flows and over time. In particular, early-stage activities such as reconnaissance are often characterized by anomalous interaction patterns rather than properties of a single connection. For example, a single source IP probing multiple ports or multiple destination hosts may indicate scanning behavior [36]. These patterns are inherently distributed and require a broader traffic context than that provided by a single flow. As a result, flow-level representations alone are insufficient to capture such global interaction patterns.

To address this limitation, and consistent with detection strategies in MITRE ATT&CK [35], we compute both source-centric and destination-centric statistical features over sliding time windows. These features capture aggregated behavior across multiple flows and complement the flow-based representation. Importantly, they can still be derived from encrypted traffic, as they rely solely on packet header information such as timestamps, IP addresses, ports, and connection metadata.

Inference Query. DEEPCHRONOS operates over specific inference queries that are constructed from flow features – including the statistical features – and the attack history. Each query is passed to the neuro-symbolic inference component, which computes the probability p over the possible query outcomes from $\{benign, phase_1, \ldots, phase_n\}$. For example, the probability of the current flow being classified as $phase_1$ is queried as:

$$phase(C_{attack}, C_{flow}, Flow, phase_1),$$

where C_{attack} and C_{flow} are the attack and flow contexts, respectively, and $Flow$ represents the flow features.

Attack Context. encodes which attack phases have been observed in the past. These are represented as binary attack-phase flags:

$$C_{attack} = \{F_1, \ldots, F_n\}$$

where $F_i = 1$ indicates that attack $phase_i$ has been already observed in the past (fed via a feed-back loop to the query construction as depicted in Figure 1), and $F_i = 0$ otherwise.

Flow Context. contains the symbolic features used to evaluate whether a flow is consistent with known attack characteristics. These include protocol information, communication direction, and service-related attributes, which are commonly used in rule-based intrusion detection systems, such as Snort [50], Suricata [43] or Zeek (formerly Bro) [45]. Formally, a flow context is defined as:

$$C_{flow} = (Protocol, Origin_{src}, Origin_{dst}, Port_{src}, Port_{dst}).$$

Flow Features. The remaining component, $Flow$, represents the statistical features extracted from the flow and which are used as input to the neural network. We exclude timestamps, IP addresses, and port numbers from these features since they are highly environment-specific and can lead to poor generalization [52, 55, 5]. In particular, attackers can easily change infrastructure, including IP addresses and port assignments, which may cause the neural model to learn patterns tied to a specific campaign rather than generalizable behavioral patterns. Instead, timestamps, IP addresses, and ports attributes are delegated to the symbolic component, which can explicitly encode and reason over them. This separation allows the neural network to focus on learning behavioral patterns that are more robust across different attack scenarios, while structural knowledge is handled through symbolic rules.

4.3 Neuro-Symbolic Attack-Phase Inference

At a high level, an attack phase $phase_i$ is inferred when its temporal prerequisites are satisfied, the observed event is consistent with predefined attack knowledge, and the neural component indicates that the current flow is malicious. This combines temporal constraints, symbolic reasoning, and neural predictions into a unified inference process. This is expressed in DEEPCHRONOS using DeepProbLog as follows:

$$\begin{aligned}
&\texttt{phase}(C_{attack}, C_{flow}, Flow, phase_i) \leftarrow \\
&\quad \texttt{plausible_attack_phase}(C_{attack} \wedge phase_i) \wedge \\
&\quad \texttt{attack_phase_rule}(C_{flow}, phase_i) \wedge \\
&\quad \texttt{neural_evidence}(Flow, phase_i).
\end{aligned}$$

`plausible_attack_phase` is a logic predicate that defines the logically consistent attack phases which can be predicted given the previously seen past attack phases (see Section 4.3.1). This rule encodes, for example, that an exfiltration must follow a compromise, otherwise it is not achievable. `attack_phase_rule` is a logic predicate defining the flow context characteristics that a traffic instance should match to be considered a plausible attack (see Section 4.3.2). This rule encode, for example, that a sadmind worm attempt must target port 111. Lastly, the `neural_evidence` represents the probability that the neural network assigns to the flow being suspicious.

On the other hand, a flow is considered benign if it does not match any of the defined attack phases. In other words, benign behavior is defined as the absence of evidence for all known attack phases and the corresponding rule is:

$$\begin{aligned}
&\texttt{phase}(C_{attack}, C_{flow}, Flow, \texttt{benign}) \leftarrow \\
&\quad \neg\Big(\texttt{attack_phase}(phase_i) \wedge \\
&\qquad \texttt{phase}(C_{attack}, C_{flow}, Flow, phase_i)\Big).
\end{aligned}$$

where $phase_i$ refers to any of the attack phases. In DEEPCHRONOS, we use the same DeepProbLog negation $\neg$ semantic expressing negation as negation-as-failure [27]. Therefore, the probability of a negated goal is given by $1-p$ and the probability of a benign flow can be expressed as:

$$P(benign) = 1 - P\left(\bigvee_{i \in \{1,\ldots,N\}} phase_i\right)$$

In other words, a flow is considered benign with high probability when the probability of all attack phases is low.

4.3.1 Temporal Constraints

Due to the step-by-step nature of MSAs, an attack phase $phase_i$ can only occur if the preceding phase $phase_{i-1}$ has been observed. DEEPCHRONOS encodes this temporal structure in the symbolic component to restrict predictions to temporally plausible attack steps.

Consider a simplified setting with three ordered phases, where $phase_1$ initiates the attack, $phase_2$ depends on $phase_1$, and $phase_3$ depends on both preceding phases. $phase_1$ is therefore always considered valid, while $phase_2$ and $phase_3$ are conditioned on the presence of earlier phases. This structure is encoded in DEEPCHRONOS using the following rules:

$$\begin{aligned}
&\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_1).\\
&\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_2) \leftarrow F_1 = 1.\\
&\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_3) \leftarrow F_1 = 1 \wedge F_2 = 1.
\end{aligned}$$

During grounding, only attack phases whose temporal constraints are satisfied will be retained. For example, given the flag configuration $(1, 0, 0)$, only $phase_1$ and $phase_2$ remain valid, while $phase_3$ is excluded due to the unmet prerequisite $F_2 = 1$.

4.3.2 Attack Phase Rules

In addition to the temporal structure of MSA, each attack step exhibits characteristic patterns in terms of network behavior and protocol usage. DEEPCHRONOS encodes these characteristics as attack-phase rules in the symbolic component of the model. The use of attack-phase rules provides several benefits. First, the symbolic component enables reasoning over features such as IP addresses and ports which are not passed to the neural component. Additionally, the rule-based structure improves interpretability, as predictions can be traced back to explicit symbolic conditions. For illustration, consider an attack $phase_i$ where an external attacker probes a victim for a vulnerable service running on port 123 using the TCP protocol. This can be expressed in DEEPCHRONOS using the following rule:

$$\begin{aligned}
&\texttt{attack_phase_rule}(Protocol, Origin_{src}, Origin_{dst},\\
&\quad Port_{dst}, phase_i) \leftarrow\\
&Protocol = \texttt{tcp} \wedge Origin_{src} = \texttt{external} \wedge\\
&Origin_{dst} = \texttt{internal} \wedge Port_{dst} = 123.
\end{aligned}$$

Using such rule, DEEPCHRONOS will not predict an attack if the port 123 is not being probed by the attacker.

4.4 Memory and Multi-Step Attack Decision

The decision and memory components decouple higher-level operational logic from the neuro-symbolic inference process. These decisions, such as when to trigger an MSA alert or how long to retain predictions in memory, do not require neuro-symbolic reasoning. On the contrary, both components are designed to be configurable by security operators to support different deployment environments.

Attack Memory. As the neuro-symbolic inference component itself is stateless, DEEPCHRONOS maintains a lightweight attack memory to preserve contextual information across flows. The memory stores previously observed attack phases for each tracked attack history state h, defined by the IP addresses involved in the attack. This ensures that retrieved attack-phase flags remain relevant to the specific network flow being analyzed. For example, assume an attacker IP_{A1} first performs a scan and later exploits a vulnerability on the target IP_{T1}. In this case, several attack-phase flags will be set for the context associated with (IP_{A1}, IP_{T1}). However, these flags should not influence inference for a different attacker IP_{A2} interacting with the network. By maintaining separate history states, the system can simultaneously track multiple attack campaigns while preventing unrelated attack histories from affecting predictions.

When an attack phase is detected, a new memory entry is created:

$$\langle h : (time_1, \texttt{attack_phase}_1, probability_1) \rangle$$

As additional attack phases are observed, the memory associated with the context is updated accordingly:

$$\begin{aligned}
\langle h : &(time_1, \texttt{attack_phase}_1, probability_1), \ldots,\\
&(time_i, \texttt{attack_phase}_i, probability_i) \rangle,
\end{aligned}$$

where, $time_i$ denotes the start time of the observed attack phase, $attack_phase_i$ represents the detected phase, and $probability_i$ corresponds to the confidence assigned by the neuro-symbolic model.

When processing a new flow, the system queries the memory using the IPs of the current flow. If a matching attack history state exists, the corresponding attack context C_{attack} is retrieved and passed to the neuro-symbolic component to guide inference. History states are maintained for a configurable retention period and are removed once they exceed the retention window without further activity. This memory component enables stateful detection of stealthy multi-step attacks that unfold gradually over extended periods of time. Additionally, it provides information about attack progression, which is useful for security analysts when deciding whether to initiate further investigation.

MSA Alarm Decision. The final alarm decision is based on the accumulation of inferred attack phases within a given attack history. Rather than triggering alarms based on isolated malicious predictions, the system considers the temporal progression of phases as evidence of an ongoing MSA. This enables the incorporation of early attack phases such as reconnaissance, which are often noisy but important for early-stage detection. Individually, such phases may produce a large number of false positives. However, by requiring multiple related attack phases to occur within the same history, many of these isolated false positives can be filtered out before an alarm is raised.

An alarm is triggered when a history h satisfies a configurable detection condition based on the set of previously observed attack phases and their associated probabilities. By default, the condition requires the observation of at least two distinct attack phases within the same history. This reduces the likelihood that noisy or isolated predictions result in unnecessary alerts. This alerting mechanism can be adapted to different operational environments by adjusting detection thresholds and phase requirements without modifying the underlying neuro-symbolic inference model.

5 Implementation

We implement DEEPCHRONOS in Python using PyTorch [44] and DeepProbLog [29]. The neural component of DEEPCHRONOS is a Long Short-Term Memory (LSTM) classifier, consisting of a single recurrent layer with 64 hidden units, followed by a fully connected layer with 32 neurons using the ReLu activation function and a final softmax layer. The architecture is intentionally kept simple to ensure that performance improvements can be more reliably attributed to the integration of symbolic reasoning rather than model complexity. Additionally, we adopt a binary classification setup, as different attack phases may exhibit similar low-level patterns and are therefore difficult to distinguish directly. The neural network is incorporated into the DeepProbLog model as a neural predicate, which outputs probabilities over the classes *benign* and *malicious*:

$$\texttt{nn}(\texttt{lstm}_{binary}, [Flow], Event, [\texttt{benign}, \texttt{malicious}])::$$
$$\texttt{neural_evidence}(Flow, Event).$$

Temporal constraints in MSA detection can be implemented with varying degrees of strictness. In particular, one may allow flexibility in the observed attack sequence – e.g., regression to earlier phases – or enforce a strictly forward progression through the attack stages. This choice directly affects the trade-off between false alarms and missed detections. More permissive rules reduce the number of missed attacks but may increase spurious transitions, while stricter rules reduce ambiguity at the cost of reduced flexibility. DEEPCHRONOS adopts a strict temporal ordering where an attack phase p_i is only considered plausible if its prerequisite phase p_{i-1} has been observed, and none of the subsequent phases p_{i+1} have yet occurred within the same context. Since the model operates on fully observed datasets with ground-truth attack progressions, the risk of missing earlier phases due to this constraint is not a concern in our setting. Formally, we encode:

$$\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_1) \leftarrow$$
$$F_1 = 0 \ \wedge \ F_2 = 0 \ \wedge \ F_3 = 0.$$
$$\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_2) \leftarrow$$
$$F_1 = 1 \ \wedge \ F_2 = 0 \ \wedge \ F_3 = 0.$$
$$\texttt{plausible_attack_phase}(F_1, F_2, F_3, phase_3) \leftarrow$$
$$F_1 = 1 \ \wedge \ F_2 = 1 \ \wedge \ F_3 = 0.$$

The temporal rules restrict inference to only two possible outcomes at a given step, i.e., the next attack phase or benign traffic. Consequently, the grounded program simplifies to

$$P(benign) = 1 - P(NextAttackPhase)$$

DEEPCHRONOS implements attack-phase rules encoding domain knowledge about the network characteristics derived from documented attack behavior, vulnerability descriptions, expert knowledge, or established IDS rulesets such as the Snort Community Rules [54, 50]. Because DEEPCHRONOS does not inspect packet payloads, payload-dependent conditions are omitted and only flow-observable properties are retained. Satisfying a rule does not independently confirm an attack phase. Instead, the rule provides symbolic evidence that is combined with the neural classifier's output during DEEPCHRONOS inference. The following examples illustrate distinct attack-phase rules implemented in DEEPCHRONOS.

Network Scanning. An inbound ICMP scan is characterized by ICMP traffic originating outside the monitored network and targeting internal hosts:

$$\begin{aligned} &scanning_rule(Protocol, Origin_{src}, Origin_{dst}) \leftarrow \\ &Protocol = icmp \wedge Origin_{src} = external \wedge \\ &Origin_{dst} = internal. \end{aligned}$$

Service Probing. The service-probing rule abstracts the header-level conditions of a Snort signature for `sadmind` probing associated with CVE-1999-0977 [39, 54]. Payload-matching conditions are excluded:

$$\begin{aligned} &probing_rule(Protocol, Origin_{src}, Origin_{dst}) \leftarrow \\ &Protocol = udp \wedge Origin_{src} = external \wedge \\ &Origin_{dst} = internal. \end{aligned}$$

DNS-Based Data Exfiltration. DNS-based exfiltration is represented by internal UDP communication directed toward a DNS service:

$$\begin{aligned} &exfiltration_rule(Protocol, Origin_{src}, Origin_{dst}, Port_{dst}) \leftarrow \\ &Protocol = udp \wedge Origin_{src} = internal \wedge \\ &Origin_{dst} = internal \wedge Port_{dst} = 53. \end{aligned}$$

Because these conditions also describe legitimate DNS traffic, the rule only establishes compatibility with the expected communication channel and complementary statistical evidence is supplied by the neural classifier.

Rule Encoding. Categorical attributes are numerically encoded before inference. Protocols use their assigned IP protocol numbers [18], while endpoint locations are represented as binary indicators, with 1 denoting internal and 0 external. For example, the probing rule is encoded as:

$$\begin{aligned} &probing_rule(Protocol, Origin_{src}, Origin_{dst}) \leftarrow \\ &Protocol = 17 \wedge Origin_{src} = 0 \wedge Origin_{dst} = 1. \end{aligned}$$

where 17 represents UDP. This encoding provides a uniform interface between flow-level data and symbolic reasoning.

6 Evaluation

This section depicts the results of DEEPCHRONOS evaluation. First, we describe our experimental settings, including datasets, evaluation metrics and baselines (Section 6.1). Then, we elaborate the effectiveness of DEEPCHRONOS in different scenarios (Section 6.2) and conduct a missed attack steps analysis and assess the tolerance of the temporal constraints (Section 6.3). Lastly, we analyze performance overhead DEEPCHRONOS performance, including latncy, throughput and peak memory usage (Section 6.4).

6.1 Experimental Setup

DEEPCHRONOS is evaluated using two publicly available MSA datasets: DARPA 2000 [34] and AIT Log Data Set V2 (AIT-LDS V2) [23, 22]. Both datasets contain network traffic packet captures (PCAPs) and labeled attack phases. The two datasets are chosen to provide complementary perspectives, combining a widely used benchmark with a more recent and realistic traffic environment. The datasets were divided into training and testing sets using a stratified 70/30 split, which ensured that samples from all attack phases were represented in both subsets. More details related to the datasets can be found in Appendix B. For both datasets, packet captures are transformed into Zeek flows, aligned with the available attack annotations, and converted into temporal neural inputs and inference queries. Full preprocessing details and the resulting class distributions are provided in Appendix C.

Metrics. DEEPCHRONOS performs a multi-class classification, and therefore several standard classification metrics are recorded, including accuracy, F1-scores (macro and weighted), false alarm rate (FAR), and detection rate (DR). In addition, per-phase F1-scores are computed. Accuracy and F1-scores are evaluated over all classes, both benign traffic and attack phases. The false alarm rate is computed as FAR = $\frac{\#\text{ benign predicted as attack}}{\#\text{ true benign samples}}$, while the detection rate is computed as DR = $\frac{\#\text{ correctly detected attacks}}{\#\text{ true attack samples}}$.

To capture the DEEPCHRONOS's ability to learn the temporal dependencies of MSA, we define two custom temporal metrics: causal violation rate (CVR) and regression violation rate (RVR). Such metrics represent two different types of temporal inconsistencies. A causal violation occurs when the model predicts attack phase p_{i+1} before phase p_i has been observed. A regression violation occurs when the model has already progressed to phase p_i but subsequently predicts an earlier attack phase. The causal violation rate is defined as CVR = $\frac{\#\text{ causal violations}}{\#\text{ total violations}}$, while the regression violation rate is defined as RVR = $\frac{\#\text{ regression violations}}{\#\text{ total violations}}$. Causal violations are considered more severe than regression violations. From a logical perspective, causal violations should not occur, as each attack step depends on the prior occurrence of prerequisite steps. Thus, CVR pinpoints instances where the detection model fails to take into account the proper attack temporal logic. Regression violations, on the other hand, may occur in practice if an attacker revisits previous attack stages. Nevertheless, they still represent temporal inconsistencies which should be possible to minimize through the incorporation of logic.

Baselines. DEEPCHRONOS is compared to both nerual and symbolic-based baselines, and state-of-the-art systems including Kitsune [33]. We implemented two neural baselines: a multiclass LSTM and an ensemble LSTM. Both models use the same underlying architecture as the neural component of DEEPCHRONOS. However, rather than performing binary classification, the baseline models directly predict the attack step through a multiclass output layer. The ensemble architecture is similar to the multiclass model but replaces the shared LSTM with multiple LSTMs (one per attack phase), whose outputs are combined through a linear prediction layer. The symbolic baselines are the DEEPCHRONOS logic programs with the neural component removed. This isolates the contribution of the logical reasoning component and provides insight into the extent to which performance can be attributed to the logic alone. Comparing these baselines with the full model allows us to assess whether the neural network provides meaningful predictive capability beyond that achieved by the logical rules.

For DEEPCHRONOS, we evaluate two variants: *(i)* an end-to-end (E2E) training version – where the neural and symbolic components are optimized jointly from scratch –, and *(ii)* its neurally pretrained (Pretrained) counterpart—where the neural component is first trained independently on the training data and subsequently integrated into the DEEPCHRONOS logic program before being fine-tuned together with the symbolic component.

Both the baseline models and the DEEPCHRONOS models are trained for a maximum of 50 epochs. The neural baselines are evaluated using the dataset-specific hyperparameter configurations selected through the ablation study in Appendix D. Specifically, we use the base feature set and a learning rate of 10^{-3} for both datasets, with temporal windows of 100 flows for AIT-LDSv2 and 10 flows for DARPA 2000.

6.2 Effectiveness

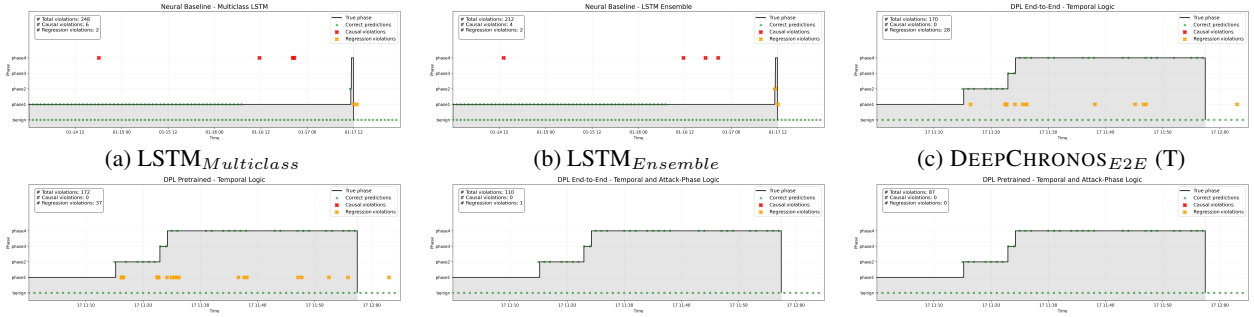
Symbolic Knowledge Study. This experiment investigates whether incorporating symbolic knowledge can help the model reliably detect individual attack steps and enforce temporally valid attack progressions. We evaluate two DEEPCHRONOS variants: *(i)* one that incorporates temporal constraints only (T), and *(ii)* another that incorporates both temporal constraints and attack-phase rules (T+R). The DEEPCHRONOS models are compared against the neural baselines, symbolic baselines, and Kitsune [33]. Table 1 depicts the results. On AIT-LDSv2, incorporating both temporal constraints and attack-phase rules (T+R) yields the strongest results, increasing macro F1 from 0.878 to 0.944 for the pretrained model and reducing its RVR from 0.152 to 0.002. On DARPA, the end-to-end T+R model achieves the best DEEPCHRONOS macro F1-score (0.940) and perfect temporal consistency ($\text{CVR} = \text{RVR} = 0$), while the neural baselines exhibit substantially more progression violations. Although Kitsune obtains the highest macro F1-score on DARPA (0.984), its FAR of 0.03719 is considerably higher than that of the end-to-end T+R model (0.00015), and, as a single-event anomaly detector, it does not provide attack-progression consistency metrics.

Neither training strategy consistently dominates across datasets. Pretraining performs best on AIT-LDSv2, while end-to-end optimization substantially improves the T+R model on DARPA. These results suggest that joint training is beneficial when the symbolic rules are strongly aligned with the observed attack behavior, whereas pretraining can be more robust when symbolic knowledge is incomplete or only weakly discriminative.

Temporal Consistency. Across both datasets, the neural baselines consistently underperform compared to the best-performing DEEPCHRONOS configurations (see Table 1). In addition, they are the only models that exhibit non-zero causal violations, suggesting weaker temporal consistency in their predictions. In contrast, the DEEPCHRONOS

Table 1: Performance and temporal consistency on AIT-LDS V2 and DARPA datasets. Results for AIT-LDS V2 are averaged over five runs, except DEEPCHRONOS$_{\text{Pretrained}}$ T, which is averaged over three runs. Results for DARPA are averaged over three runs. Standard deviations are reported only for macro F1. Models are ranked by macro F1-score within each dataset. Best results for CVR and RVR within each dataset are highlighted in bold.

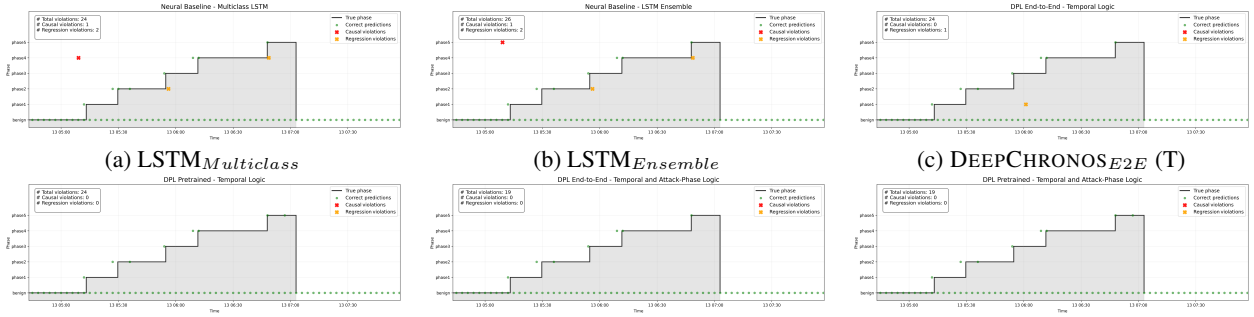
Model	Macro F1 ($\pm$ std)	Weighted F1	Accuracy	DR	FAR	CVR	RVR
AIT-LDS V2							
DEEPCHRONOS$_{\text{Pretrained}}$ T+R	0.944 ± 0.024	0.999	0.999	0.990	0.0003	**0.000**	**0.002**
DEEPCHRONOS$_{\text{E2E}}$ T+R	0.940 ± 0.010	0.999	0.999	0.991	0.0005	**0.000**	0.014
LSTM$_{\text{Multiclass}}$	0.880 ± 0.015	0.998	0.998	0.988	0.0018	0.019	0.021
DEEPCHRONOS$_{\text{Pretrained}}$ T	0.878 ± 0.020	0.998	0.998	0.983	0.0008	**0.000**	0.152
LSTM$_{\text{Ensemble}}$	0.858 ± 0.024	0.998	0.998	0.991	0.0019	0.027	0.017
DEEPCHRONOS$_{\text{E2E}}$ T	0.842 ± 0.013	0.998	0.998	0.981	0.0008	**0.000**	0.180
Kitsune [33]	0.645	0.938	0.947	0.231	0.0125	–	–
Logic$_{\text{Baseline}}$ T+R	0.631 ± 0.038	0.917	0.925	0.714	0.0627	**0.000**	0.047
Logic$_{\text{Baseline}}$ T	0.092 ± 0.000	0.100	0.053	1.000	1.000	**0.000**	0.200
DARPA							
Kitsune [33]	0.984	0.984	0.984	0.999	0.03719	–	–
DEEPCHRONOS$_{\text{E2E}}$ T+R	0.940 ± 0.014	0.999	0.999	0.999	0.00015	**0.000**	**0.000**
DEEPCHRONOS$_{\text{Pretrained}}$ T+R	0.892 ± 0.006	0.999	0.999	0.999	0.00009	**0.000**	**0.000**
Logic$_{\text{Baseline}}$ T+R	0.891 ± 0.079	0.963	0.962	0.999	0.09021	**0.000**	0.001
DEEPCHRONOS$_{\text{Pretrained}}$ T	0.872 ± 0.013	0.999	0.999	0.999	0.00021	**0.000**	**0.000**
DEEPCHRONOS$_{\text{E2E}}$ T	0.839 ± 0.019	0.999	0.999	0.999	0.00015	**0.000**	0.014
LSTM$_{\text{Multiclass}}$	0.827 ± 0.016	0.999	0.999	0.999	0.00030	0.027	0.081
LSTM$_{\text{Ensemble}}$	0.820 ± 0.014	0.999	0.999	0.999	0.00032	0.053	0.092
Logic$_{\text{Baseline}}$ T	0.558 ± 0.000	0.655	0.585	1.000	0.99834	**0.000**	0.144



(a) LSTM$_{Multiclass}$ (b) LSTM$_{Ensemble}$ (c) DEEPCHRONOS$_{E2E}$ (T)

(d) DEEPCHRONOS$_{Pretrained}$ (T) (e) DEEPCHRONOS$_{E2E}$ (T+R) (f) DEEPCHRONOS$_{Pretrained}$ (T+R)

Figure 2: Temporal consistency comparison across neural and neuro-symbolic models on AIT-LDS V2.

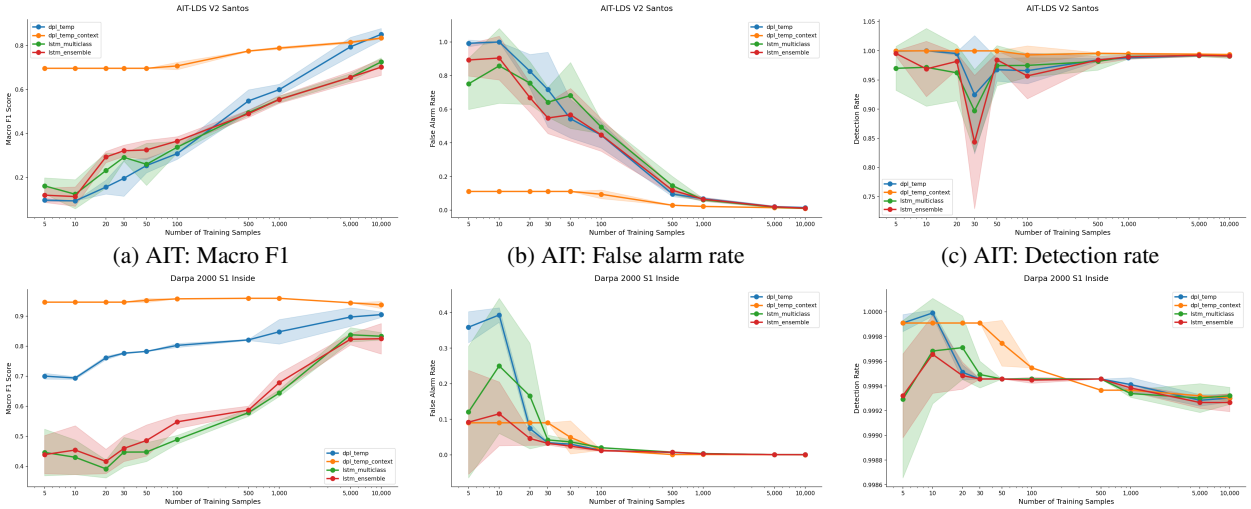


(a) LSTM$_{Multiclass}$ (b) LSTM$_{Ensemble}$ (c) DEEPCHRONOS$_{E2E}$ (T)

(d) DEEPCHRONOS$_{Pretrained}$ (T) (e) DEEPCHRONOS$_{E2E}$ (T+R) (f) DEEPCHRONOS$_{Pretrained}$ (T+R)

Figure 3: Temporal consistency comparison across neural and neuro-symbolic models on DARPA 2000.

models substantially reduce both causal and regression violations, indicating improved temporal consistency under the proposed modeling constraints. Figures 2 and 3 further illustrate these effects. Temporal logic enforces strict ordering constraints that eliminate causal violations across all settings. The inclusion of attack-phase rules further reduces regression violations, resulting in more temporally consistent predictions overall. Since Kitsune is an anomaly detector that only distinguishes between benign and anomalous traffic, temporal consistency metrics are not applicable and are therefore omitted.



(a) AIT: Macro F1 (b) AIT: False alarm rate (c) AIT: Detection rate

(d) DARPA: Macro F1 (e) DARPA: False alarm rate (f) DARPA: Detection rate

Figure 4: Effect of training set size on model performance across AIT-LDS V2 and DARPA 2000. Results show how macro F1, FAR, and DR vary as the number of training samples increases for different model configurations.

Takeaways

Temporal and attack-phase knowledge. Incorporating temporal and attack-phase knowledge improves attack-step detection performance while substantially reducing temporal inconsistencies. The strongest results are consistently obtained when both knowledge sources are combined.

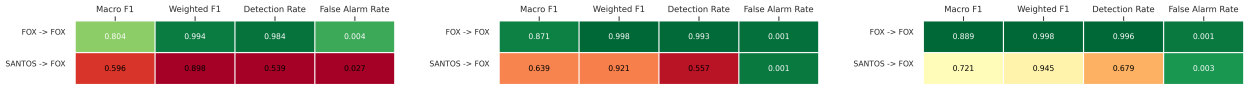
End-to-end learning. Joint optimization is most beneficial when temporal constraints are complemented by informative attack-phase rules that closely align with the observed attack behavior. Otherwise, pretraining may provide more stable performance.

Training Data Scarsity. We here evaluate how the performance of DEEPCHRONOS models and neural baselines varies under different levels of training data availability. To ensure balanced learning conditions, we construct training subsets with an equal number of benign and attack samples per attack step. For each dataset size, results are averaged over five independent runs per model. Figures 4a to 4c shows results for AIT-LDS V2 and Figures 4d to 4f for DARPA 2000. Across both datasets, the DEEPCHRONOS model with temporal and attack-phase knowledge consistently achieves strong performance in low-data regimes. In contrast, temporal knowledge alone exhibits dataset-dependent behavior. On DARPA it yields strong initial performance, while on AIT-LDS V2 it leads to slower early convergence. As the number of training samples increases, the neural baselines gradually approach the performance of the DEEPCHRONOS model with temporal and attack-phase knowledge. This convergence occurs at approximately 10,000 samples for AIT-LDS V2 and 5,000 samples for DARPA 2000. However, even in higher data regimes, models incorporating symbolic knowledge maintain a consistent advantage in macro F1. The error bars indicate that the DEEPCHRONOS model with temporal and attack-phase knowledge exhibits the most stable learning behavior across runs. In contrast, neural baselines show higher variance across all data regimes. On AIT-LDS V2, minor fluctuations are observed for the DEEPCHRONOS model with temporal and attack-phase knowledge at around 100 samples. A similar effect appears on DARPA at approximately 50 samples. The temporal-only DEEPCHRONOS model shows increased variability across most sample sizes on AIT-LDS V2, while on DARPA higher variance in macro F1 emerges beyond approximately 1,000 samples.

Takeaways

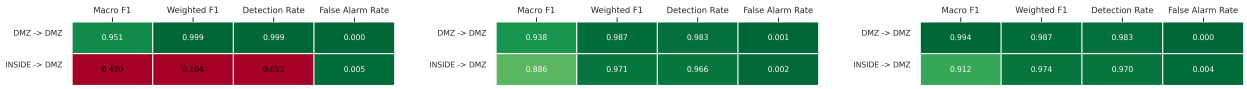
Data efficiency. Temporal and attack-phase symbolic knowledge improves data efficiency, enabling strong performance in low-data regimes and reducing the number of training samples required compared to purely neural baselines.

Generalizability. We analyze whether DEEPCHRONOS generalizes beyond the MSA scenarios it is trained on. In particular, we investigate whether incorporating symbolic knowledge enables the model to learn representations that



(a) Multiclass LSTM (b) DEEPCHRONOS Pretrained (c) DEEPCHRONOS End-to-End

Figure 5: Confusion matrices illustrating cross-scenario generalization on AIT-LDSv2. Each matrix compares performance when the model is trained and tested on the same scenario against performance when the model is evaluated on a different scenario.



(a) Multiclass LSTM (b) DEEPCHRONOS Pretrained (c) DEEPCHRONOS End-to-End

Figure 6: Confusion matrices illustrating cross-sensor generalization on DARPA 2000. Each matrix compares performance when the model is trained and tested on the same sensor against performance when it is evaluated on traffic from the other sensor.

transfer across attack variants or network vantage points, rather than overfitting to dataset-specific patterns. We focus on the DEEPCHRONOS model with both temporal and attack-phase knowledge—i.e., DEEPCHRONOS (T+R).

We consider two cross-domain evaluation settings. For AIT-LDS V2, the model is trained on one MSA scenario (Santos) and evaluated on a different scenario (Fox), allowing us to assess generalization across variations of the same MSA setting. For DARPA 2000, the model is trained on the Inside dataset and evaluated on the DMZ dataset, enabling evaluation of generalization across different network vantage points. In both settings, models are trained on the full source scenario (Santos or Inside) and evaluated on the complete target scenario (Fox or DMZ). We compare both end-to-end training and a pretrained configuration for the neural component, and benchmark against a multiclass neural baseline.

Due to computational constraints, a window size of 10 was used for AIT-LDS V2 in this experiment instead of the default value of 100. This introduces a reduced temporal context compared to previous experiments. As a result, absolute performance levels should be interpreted with caution, and comparisons should primarily focus on relative differences between models within this setting.

Figure 5 presents the results of generalization across scenarios on AIT-LDS V2. The neural baseline exhibits limited generalization across attack scenarios, whereas incorporating temporal and attack-phase knowledge leads to substantial improvements across all evaluation metrics. This indicates that symbolic constraints introduce useful inductive biases that transfer across attack variants. The end-to-end DEEPCHRONOS model achieves the strongest overall performance, suggesting that learned representations benefit from direct interaction with symbolic constraints during training. A phase-level breakdown is provided in Table 2. All models struggle to detect attack phase 4 when trained on Santos and evaluated on Fox, indicating that this phase exhibits substantial variation across scenarios. Nevertheless, the DEEPCHRONOS models consistently outperform both the neural and symbolic baselines across most attack phases.

Figure 6 presents the results of the cross-vantage point experiment. Similar to the cross-scenario setting, the neural baseline shows limited generalization capability, whereas the DEEPCHRONOS models achieve substantially higher macro F1 and weighted F1-scores. The highest macro F1-score is obtained by the symbolic baseline. However, this comes at the cost of a high false alarm rate, as reflected in the poor F1-score for the benign class (see Table 3). In contrast, the DEEPCHRONOS models achieve a more balanced performance profile, while maintaining competitive overall performance. Lastly, the neural baseline struggles across multiple phases, indicating less stable performance across attack stages.

Takeaways

Generalization. Symbolic temporal and attack-phase knowledge improves generalization across both attack scenarios and network vantage points. The strongest results are consistently obtained when symbolic constraints are integrated jointly with neural learning, indicating that interaction during training is beneficial for transferability across attack variants.

Table 2: Per-phase generalizability performance on AIT-LDS V2. The models are trained on Santos and tested on Fox. The best performing score per metric is in bold.

Model	Macro F1	Weighted F1	$F1_{Benign}$	$F1_{P1}$	$F1_{P2}$	$F1_{P3}$	$F1_{P4}$
$DEEPCHRONOS_{E2E}$	**0.721**	0.945	**0.997**	0.977	0.752	**0.877**	0.026
$DEEPCHRONOS_{Pretrained}$	0.639	0.921	0.962	**0.981**	0.620	0.585	**0.046**
$LSTM_{Multiclass}$	0.596	0.898	**0.997**	0.965	0.534	0.511	0.024
$Logic_{Baseline}$	0.561	**0.953**	0.982	0.000	**0.999**	0.824	0.000

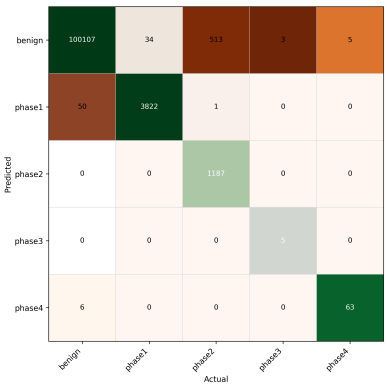
Table 3: Per-phase generalizability performance on DARPA 2000. The models are trained on INSIDE and tested on DMZ. The best performing score per metric is in bold.

Model	Macro F1	Weighted F1	$F1_{Benign}$	$F1_{P1}$	$F1_{P2}$	$F1_{P3}$	$F1_{P4}$	$F1_{P5}$
$Logic_{Baseline}$	**0.931**	0.871	0.686	**0.998**	**1.000**	**1.000**	**0.974**	0.926
$DEEPCHRONOS_{E2E}$	0.912	**0.974**	**0.951**	0.586	**1.000**	**1.000**	0.947	0.990
$DEEPCHRONOS_{Pretrained}$	0.886	0.971	0.947	0.431	**1.000**	0.974	0.973	**0.991**
$LSTM_{Multiclass}$	0.410	0.104	0.380	0.468	0.367	0.904	0.333	0.007

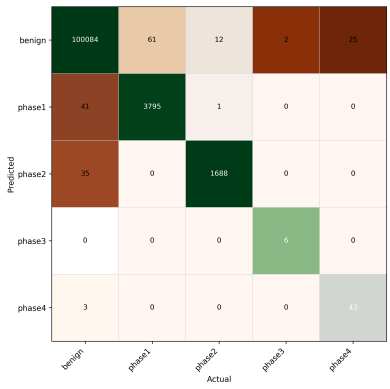
6.3 Missed Attack Steps

All previous experiments assume perfect knowledge of previously observed attack phases. In practice, however, intrusion detection systems may fail to detect certain attack steps, resulting in incomplete attack histories. Therefore, we here investigate whether the temporal constraints can be revised to tolerate such missing observations. More specifically, we evaluate whether the model can still infer $phase_i$ even when the preceding phase $phase_{i-1}$ has not been observed. We relax the temporal constraint for $phase_2$ over the AIT-LDS V2 dataset, allowing the model to predict the second attack phase even when the first phase has not been observed.

To simulate missed detections, we introduce noise by randomly resetting all attack-phase flags to 0 for 30% of the true $phase_2$ samples in both the training and test sets. This removes the temporal evidence that would normally permit the prediction of $phase_2$. Table 4 summarizes the overall and per-phase performance. Compared to the original setting, the revised logic evaluated on the corrupted dataset exhibits a modest reduction in overall performance. Nevertheless, it substantially outperforms the original logic when applied to the same corrupted data, improving both the macro F1-score and the F1-score associated with $phase_2$. The confusion matrices provide further insight into this behavior. As depicted in Figure 7b, the revised logic misses only 13 $phase_2$ flows despite the corrupted attack phase flags. In contrast, the original logic fails to recover these missing phases, resulting in 514 misclassified $phase_2$ flows (Figure 7a).



(a) Detection performance of DEEPCHRONOS using the original logic.



(b) Detection performance of DEEPCHRONOS using the revised temporal logic.

Figure 7: Confusion matrices for the $DEEPCHRONOS_{E2E}$ model on the currepted AIT-LDS V2 dataset. Relaxing the temporal constraints used in DEEPCHRONOS's symbolic knowledge defines a flexible detector that can identify attacks even when some phases observations are missed.

Table 4: Performance under missing attack phases in AIT-LDS V2. The best-performing value for each metric is highlighted in bold.

Configuration	Macro F1	Weighted F1	$F1_{Benign}$	$F1_{P1}$	$F1_{P2}$	$F1_{P3}$	$F1_{P4}$
$Logic_{Orig}$, $Data_{Orig}$	**0.931**	**0.999**	**0.999**	0.988	**1.000**	0.769	0.900
$Logic_{Orig}$, $Data_{Corr}$	0.899	0.994	0.997	**0.989**	0.822	0.769	**0.920**
$Logic_{Rev}$, $Data_{Corr}$	0.917	0.998	**0.999**	0.987	0.986	**0.857**	0.754

Table 5: Runtime efficiency on the AIT-LDS V2 test set. Latency is averaged per prediction across several runs, and throughput is computed as $1/\text{latency}$. The best-performing value for each metric is highlighted in bold.

Model	Latency (ms)	Throughput (pred/s)	Peak memory (GB)
DEEPCHRONOS$_{Pretrained}$ T	2.48 ± 40.28	403	$\mathbf{25.90 \pm 0.57}$
DEEPCHRONOS$_{E2E}$ T	2.41 ± 36.93	415	29.83 ± 0.56
DEEPCHRONOS$_{Pretrained}$ T+R	0.46 ± 3.32	2155	30.19 ± 0.01
DEEPCHRONOS$_{E2E}$ T+R	0.46 ± 1.09	2152	30.19 ± 0.00
$Logic_{Baseline}$ T+R	0.23 ± 15.03	4354	30.13 ± 0.01
$Logic_{Baseline}$ T	$\mathbf{0.14 \pm 0.35}$	**6985**	29.95 ± 0.03

Takeaways

Robustness to missing observations. Relaxing temporal constraints improves robustness to missing attack observations, while preserving overall detection performance.

6.4 Performance Overhead

To evaluate the practical feasibility of the proposed system, we measure average inference latency per prediction, throughput, and peak memory usage. Inference time corresponds to the full execution of a DEEPCHRONOS query, including both logical grounding and neural forward passes when required. System metrics were collected using `psutil` [49] during training and inference.

AIT-LDS V2. The results in Table 5 show that mean inference latency ranges from 0.14 ms to 2.48 ms across models, corresponding to throughput values between approximately 400 and 7,000 predictions per second. The symbolic baselines achieve the lowest latency, followed by the neuro-symbolic models with temporal and attack-phase knowledge. These DEEPCHRONOS models achieve substantially lower average latency compared to their temporal-only counterparts. Peak memory usage remains relatively stable across all configurations, ranging from approximately 25.9 GB to 30.2 GB. The small variation across models further suggests that memory consumption is largely driven by the shared execution infrastructure rather than the specific reasoning configuration.

DARPA 2000. As shown in Table 6, mean inference latency ranges from 2.19 ms to 5.60 ms. As in the AIT-LDS V2 setting, the symbolic baselines are the fastest, while neuro-symbolic models incur additional computational overhead due to neural execution. Overall, DARPA 2000 exhibits higher inference latency and a smaller performance gap between models compared to AIT-LDS V2. Peak memory usage is again stable across models, ranging from approximately 12.6 GB to 15.3 GB, with only minor differences between configurations. This suggests that memory consumption scales primarily with dataset size rather than the inclusion of symbolic reasoning components.

Takeaways

Inference latency and memory consumption. The proposed neuro-symbolic approach achieves low inference latency and stable memory consumption.

7 Discussions and Limitations

Complementary Neural and Symbolic Reasoning. The results show that neural learning and symbolic reasoning provide complementary evidence for MSA detection. Attack-phase rules assess whether flows exhibit phase-compatible characteristics, while temporal constraints enforce consistency with the observed attack progression. Sym-

Table 6: Runtime efficiency on the DARPA 2000 test set. Latency is averaged per prediction across several runs, and throughput is computed as $1/\text{latency}$. The best-performing value for each metric is highlighted in bold.

Model	Latency (ms)	Throughput (pred/s)	Peak memory (GB)
DEEPCHRONOS$_{\text{Pretrained}}$ T	5.28 ± 36.53	189	15.32 ± 0.02
DEEPCHRONOS$_{\text{E2E}}$ T	5.34 ± 34.69	187	15.31 ± 0.00
DEEPCHRONOS$_{\text{Pretrained}}$ T+R	5.60 ± 38.83	178	15.19 ± 0.00
DEEPCHRONOS$_{\text{E2E}}$ T+R	5.29 ± 40.86	189	15.20 ± 0.01
Logic$_{\text{Baseline}}$ T+R	3.06 ± 31.49	327	$\mathbf{12.65 \pm 0.64}$
Logic$_{\text{Baseline}}$ T	$\mathbf{2.19 \pm 22.00}$	**457**	12.99 ± 0.01

bolic reasoning is particularly effective on DARPA 2000, whose phases have well-defined network characteristics, whereas neural learning is more important on AIT-LDSv2, where attack and benign behavior overlap. Their combination generally performs best because learned patterns compensate for incomplete rules, while symbolic knowledge suppresses implausible predictions. This benefit nevertheless depends on rule quality. Restrictive rules may exclude valid variants, whereas broad rules provide limited discrimination.

Temporal Reasoning, Data Efficiency, and Attack History. DEEPCHRONOS separates local flow classification from long-range progression reasoning: the LSTM models recent traffic, while the symbolic history relates predictions across longer periods. This allows dataset-specific neural windows –100 flows for AIT-LDSv2 and 10 for DARPA 2000 – without limiting the duration modeled symbolically, and is particularly beneficial when labeled data are scarce. Similar distinctions between low-level evidence and higher-level attack stories appear in SLEUTH, HOLMES, and ATLAS [16, 32, 2]. However, the evaluation assumes one predefined progression and uses ground-truth phases to update the history. It therefore excludes phase reordering, interleaved attacks, competing hypotheses, and error propagation from previous predictions. The results consequently quantify the value of temporal knowledge under controlled-history conditions rather than fully online attack reconstruction.

Training Strategy and Computational Cost. Pretrained DEEPCHRONOS models demonstrate that symbolic knowledge improves inference even when introduced after neural training, while the stronger cross-dataset performance of end-to-end models suggests that joint optimization better aligns learned representations with the logic program. Symbolic inference introduces modest overhead, and attack-phase rules can reduce latency by excluding incompatible flows before neural evaluation. Nevertheless, the measurements cover model inference rather than the complete operational pipeline; online flow extraction, query construction, state management, larger rule bases, concurrent attacks, and sustained traffic may introduce additional costs.

Observability, Generalization, and Evasion. The evaluation uses network-flow metadata from two controlled benchmarks and therefore captures only the network-observable portion of an attack. Host-local actions, including privilege escalation, credential access, persistence, and command execution, may require richer telemetry; provenance-based systems such as HOLMES and UNICORN obtain such evidence from host auditing at the cost of more invasive data collection [32, 14]. Moreover, performance on DARPA 2000 and AIT-LDSv2 does not establish effectiveness on operational traffic with different base rates, environments, and attacker behavior [55]. Adaptive attackers may also manipulate flow characteristics, reorder phases, insert benign activity, or contaminate the stored history. Practical evasion and mimicry attacks against learned and provenance-based detectors have been demonstrated [47, 12]. Robustness against such behavior therefore requires dedicated adversarial evaluation.

8 Related Work

Prior MSA defenses largely follow a *detect first, correlate later* paradigm where IDS first identify suspicious events, after which a separate stage links them into an attack scenario [40].

Detect First, Correlate Later. Classical alert-correlation methods link IDS outputs through shared attributes, causal dependencies, or attack graphs. Ning et al. connect alerts when the consequences of one action satisfy the prerequisites of another and later hypothesize steps missed by the IDS [41, 42]. Angelini et al. detect online progress along known attack-graph paths, while M^{2}ASK maps NIDS alerts to MITRE ATT&CK and correlates them into multi-step scenarios [3, 31]. These systems may operate online, but progression reasoning remains downstream of alert generation and therefore cannot revise the detection of the underlying events.

Provenance-based systems similarly reason over host audit events after their collection. For example, HOLMES correlates suspicious information flows into high-level APT scenarios [32], while UNICORN detects anomalous long-running provenance graphs [14], and MAGIC uses self-supervised masked graph learning for multi-granularity APT

detection and concept-drift adaptation [19]. NODLINK performs online APT detection by screening provenance events and approximating a Steiner tree that recovers a concise attack-related subgraph [24]. NODLINK is closest to DEEPCHRONOS in targeting timely detection of evolving attacks, but differs in observability and output as it requires endpoint provenance and reconstructs a causal investigation graph, whereas DEEPCHRONOS uses network flows and returns progression-consistent attack phase alerts. The approaches are therefore complementary rather than directly interchangeable: provenance provides richer host-level causality, while flow-level detection avoids endpoint instrumentation and remains applicable to encrypted network traffic.

Learning Attack Progression. Data-driven IDSs such as DeepLog and Kitsune learn anomalous behavior from logs or flows, but do not explicitly represent an attack's phase progression [9, 33]. More recent systems aim to model longer contexts. For example, [11] predicts APT progression from specialized detector outputs, while StageFinder estimates ATT&CK stages from provenance graphs [46], and Tracegram learns temporal dependencies among ordered flows [48]. These methods demonstrate the value of temporal context, but either depend on upstream detections or learn progression statistically without enforcing explicit phase prerequisites.

DEEPCHRONOS overcomes these limitations through *detection through logical progression*. Instead of learning attack progression statistically, DEEPCHRONOS embeds explicit attack-phase rules and temporal prerequisites directly into flow-level inference using neuro-symbolic reasoning. Progression knowledge is therefore explicitly encoded and constrains each detection decision with complete trustworthiness.

9 Conclusions

We presented DEEPCHRONOS, a neuro-symbolic NIDS that embeds temporal and attack-phase knowledge directly into attack detection. By combining neural classification, probabilistic symbolic reasoning, and context-specific attack memory, DEEPCHRONOS produces predictions that are both data-driven and consistent with the modeled attack progression. Evaluation on DARPA 2000 and AIT-LDS V2 shows that DEEPCHRONOS improves attack-phase classification over neural baselines, eliminates causal violations, and improves data efficiency and cross-domain generalization. These results demonstrate the potential of neuro-symbolic reasoning to support more coherent and operationally useful detection of multi-step attacks.

References

[1] Andrea Agiollo, Enkeleda Bardhi, Mauro Conti, Riccardo Lazzeretti, Eleonora Losiouk, and Andrea Omicini. Gnn4ifa: Interest flooding attack detection with graph neural networks. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pages 615–630. IEEE, 2023.

[2] Abdulellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z. Berkay Celik, Xiangyu Zhang, and Dongyan Xu. ATLAS: A sequence-based learning approach for attack investigation. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 3005–3022. USENIX Association, 2021.

[3] Marco Angelini, Silvia Bonomi, Emanuele Borzi, Antonella Del Pozzo, Simone Lenti, and Giuseppe Santucci. An attack graph-based on-line multi-step attack detector. In *Proceedings of the 19th International Conference on Distributed Computing and Networking*, 2018.

[4] Enkeleda Bardhi, Chenxing Ji, Ali Imran, Muhammad Shahbaz, Riccardo Lazzeretti, Mauro Conti, and Fernando Kuipers. O'mine: A novel collaborative ddos detection mechanism for programmable data-planes. In *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*, pages 771–788. IEEE, 2025.

[5] Marco Cantone, Claudio Marrocco, and Alessandro Bria. Machine learning in network intrusion detection: A cross-dataset generalization study. *IEEE Access*, 12:144489–144508, 2024.

[6] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, Doha, Qatar, 2014. Association for Computational Linguistics.

[7] Giovanni Ciatto, Federico Sabbatini, Andrea Agiollo, Matteo Magnini, and Andrea Omicini. Symbolic knowledge extraction and injection with sub-symbolic predictors: A systematic literature review. *ACM Computing Surveys*, 56(6):1–35, 2024.

[8] Luc De Raedt, Angelika Kimmig, Hannu Toivonen, and M. Veloso. ProbLog: A probabilistic prolog and its application in link discovery. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pages 2462–2467, 2007.

[9] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298, 2017.

[10] Artur S. d'Avila Garcez and Luis C. Lamb. Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, 56:12387–12406, 2023.

[11] Ibrahim Ghafir, Mohammad Hammoudeh, Vaclav Prenosil, Liangxiu Han, Robert Hegarty, Khaled Rabie, and Francisco J. Aparicio-Navarro. Detection of advanced persistent threat using machine-learning correlation analysis. *Future Generation Computer Systems*, 89:349–359, 2018.

[12] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems. In *30th Annual Network and Distributed System Security Symposium*. The Internet Society, 2023.

[13] Steffen Haas and Mathias Fischer. On the alert correlation process for the detection of multi-step attacks and a graph-based realization. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, pages 1697–1706, 2019.

[14] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. UNICORN: Runtime provenance-based detector for advanced persistent threats. In *Proceedings of the Network and Distributed System Security Symposium*, 2020.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[16] Md Nahid Hossain, Sadegh M. Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R. Sekar, Scott D. Stoller, and V. N. Venkatakrishnan. SLEUTH: Real-time attack scenario reconstruction from COTS audit data. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 487–504, Vancouver, BC, 2017. USENIX Association.

[17] Eric M. Hutchins, Michael J. Cloppert, and Rohan M. Amin. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. In *Proceedings of the International Conference on Information Warfare and Security*, pages 113–125, 2011.

[18] Internet Assigned Numbers Authority. Protocol numbers, n.d.

[19] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. MAGIC: Detecting advanced persistent threats via masked graph representation learning. In *33rd USENIX Security Symposium*. USENIX Association, 2024.

[20] JumpCloud. What is a network flow?, 2025. Updated: 2025-07-22.

[21] Max Landauer, Florian Skopik, Maximilian Frank, Wolfgang Hotwagner, Markus Wurzenberger, and Andreas Rauber. AIT log data set v2.0, 2022. Accessed: 2026-06-15.

[22] Max Landauer, Florian Skopik, Maximilian Frank, Wolfgang Hotwagner, Markus Wurzenberger, and Andreas Rauber. Maintainable log datasets for evaluation of intrusion detection systems. *IEEE Transactions on Dependable and Secure Computing*, 20(4):3466–3482, 2023.

[23] Max Landauer, Florian Skopik, Markus Wurzenberger, Wolfgang Hotwagner, and Andreas Rauber. Have it your way: Generating customized log datasets with a model-driven simulation testbed. *IEEE Transactions on Reliability*, 70(1):402–415, 2021.

[24] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. NODLINK: An online system for fine-grained APT attack detection and investigation. In *Proceedings of the 2024 Network and Distributed System Security Symposium*. Internet Society, 2024.

[25] Richard P. Lippmann, David J. Fried, Isaac Graf, Joshua W. Haines, Kristopher R. Kendall, David McClung, Dan Weber, Seth E. Webster, Dan Wyschogrod, Robert K. Cunningham, and Marc A. Zissman. Evaluating intrusion detection systems: The 1998 DARPA off-line intrusion detection evaluation. In *Proceedings DARPA Information Survivability Conference and Exposition*, volume 2, pages 12–26, 2000.

[26] Lockheed Martin. Cyber kill chain, n.d. Accessed: 2026-06-15.

[27] Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

[28] Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Neural probabilistic logic programming in DeepProbLog. *Artificial Intelligence*, 298:103504, 2021.

[29] Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Neural probabilistic logic programming in deepproblog. *Artificial Intelligence*, 298:103504, 2021.

[30] Giuseppe Marra, Sebastijan Dumančić, Robin Manhaeve, and Luc De Raedt. From statistical relational to neuro-symbolic artificial intelligence: A survey. *Artificial Intelligence*, 328:104062, 2024.

[31] Qiaoran Meng, Nay Oo, Yuning Jiang, Hoon Wei Lim, and Biplab Sikdar. Poster: M^{2}ASK: A correlation-based multi-step attack scenario detection framework using MITRE ATT&CK mapping. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, pages 4979–4981, 2024.

[32] Sadegh M. Milajerdi, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. HOLMES: Real-time APT detection through correlation of suspicious information flows. In *2019 IEEE Symposium on Security and Privacy*, pages 1137–1152, 2019.

[33] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium*, 2018.

[34] MIT Lincoln Laboratory. 2000 DARPA intrusion detection scenario specific datasets, n.d.

[35] MITRE ATT&CK. Enterprise matrix, n.d. Accessed: 2026-06-15.

[36] MITRE ATT&CK. Reconnaissance, n.d. Accessed: 2026-06-15.

[37] MITRE Corporation. Solarwinds compromise, campaign C0024, n.d. Accessed: 2026-08-16.

[38] Azqa Nadeem, Sicco E. Verwer, Stephen Moskal, and Shanchieh Jay Yang. Alert-driven attack graph generation using S-PDFA. *IEEE Transactions on Dependable and Secure Computing*, 19(2):731–746, 2022.

[39] National Institute of Standards and Technology. CVE-1999-0977 detail, n.d.

[40] Julio Navarro, Aline Deruyver, and Pierre Parrend. A systematic survey on multi-step attack detection. *Computers & Security*, 76:214–249, 2018.

[41] Peng Ning, Yun Cui, and Douglas S. Reeves. Constructing attack scenarios through correlation of intrusion alerts. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pages 245–254, 2002.

[42] Peng Ning, Dingbang Xu, Christopher G. Healey, and Robert St. Amant. Building attack scenarios through integration of complementary alert correlation methods. In *Proceedings of the Network and Distributed System Security Symposium*, pages 97–111, 2004.

[43] Open Information Security Foundation. Suricata: Open source IDS, IPS, and network security monitoring engine. Software repository; accessed: 2026-08-10.

[44] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[45] Vern Paxson. Bro: A system for detecting network intruders in real-time. *Computer Networks*, 31(23–24):2435–2463, 1999.

[46] Trung V. Phan and Thomas Bauschert. Learning the APT kill chain: Temporal reasoning over provenance data for attack stage estimation. In *IEEE Global Communications Conference*, 2026. Accepted for publication.

[47] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. Intriguing properties of adversarial ML attacks in the problem space. In *2020 IEEE Symposium on Security and Privacy*, pages 1332–1349. IEEE, 2020.

[48] Jian Qu, Yuchen Zhang, Jialong Zhang, Jianfeng Li, and Xiaobo Ma. Tracegram: Framing trace-level traffic analysis with temporally-aware multiple instance learning. In *35th USENIX Security Symposium*, 2026.

[49] Giampaolo Rodola. psutil: Cross-platform library for process and system monitoring, 2009.

[50] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration*, LISA '99, pages 229–238, Seattle, Washington, USA, 1999. USENIX Association.

[51] Raoul Saad, Enkeleda Bardhi, Andrea Agiollo, and Fernando Kuipers. Deepweave: Weaving neural and symbolic evidence for ml-based network intrusion detection system. *arXiv preprint arXiv:2610.41541*, 2026.

[52] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection. *Big Data Research*, 30:100359, 2022.

[53] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (IDPS). Technical Report Special Publication 800-94, National Institute of Standards and Technology, 2007.

[54] Snort. Rule downloads, n.d.

[55] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010.

[56] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019.

[57] Xiaoyu Wang, Xiaorui Gong, Lei Yu, and Jian Liu. MAAC: Novel alert correlation method to detect multi-step attack. *IEEE Access*, 9:93813–93827, 2021.

[58] Andrew Witty. Testimony of andrew witty, chief executive officer, unitedhealth group, before the senate finance committee: Hacking america's health care: Assessing the change healthcare cyber attack and what's next. Testimony before the United States Senate Committee on Finance, May 2024. Accessed: 2026-08-16.

A Use of Generative AI

Generative AI tools – mainly GPT 5.6 – were used to improve the English language, clarity, and readability of the manuscript. Their use was limited primarily to grammar correction, sentence restructuring, and stylistic refinement. The authors developed the research methodology, implemented the system, conducted and interpreted the experiments, and wrote the substantive content of the manuscript. All AI-assisted text was reviewed, revised, and approved by the authors, who take full responsibility for the final content.

B Dataset Details

B.1 DARPA 2000 Intrusion Detection Dataset

DARPA 2000 was produced by MIT Lincoln Laboratory and captures network traffic from a simulated Air Force network. Packet captures are available from sensors located inside the network and in the demilitarized zone (DMZ). The dataset contains two distributed denial-of-service (DDoS) scenarios, i.e., LLDOS 1.0, executed by a novice attacker, and the stealthier LLDOS 2.0.2 scenario. Because LLDOS 1.0 contains substantially more attack observations, we use its inside and DMZ traffic for training and evaluation. The LLDOS 1.0 attack consists of five phases [34]:

1. **Scanning:** ICMP Echo Requests are used to discover active hosts.
2. **Probing:** `sadmind` "ping" requests identify Solaris hosts running the vulnerable service.
3. **Exploitation:** the attacker exploits the `sadmind` vulnerability, CVE-1999-0977 [39], to compromise selected hosts.
4. **Installation:** the `mstream` DDoS software is installed on the compromised hosts.
5. **DDoS:** the compromised hosts launch a distributed denial-of-service attack against an external target.

B.2 AIT Log Dataset V2

AIT Log Dataset V2 (AIT-LDSv2) comprises eight synthetic datasets collected from enterprise-like testbeds at the Austrian Institute of Technology in 2022 [23, 22]. Each dataset covers four to six days of benign activity and a multi-step attack executed during the observation period. The testbeds contain multiple hosts, services, and users, with network traffic and host-level logs collected throughout the environment. The attack comprises six phases [23, 22]:

1. **Scanning:** host discovery, DNS brute-force, service scanning, directory enumeration, and WordPress scanning are performed using tools including Nmap, Dirb, and WPScan.
2. **Exploitation:** CVE-2020-24186 is exploited to upload a PHP webshell, which is subsequently used for host discovery and command execution.
3. **Password Cracking:** password hashes are attacked using John the Ripper and a common-password wordlist.

Table 7: Selected AIT-LDS V2 scenarios. Adapted from [21].

Scenario	Simulation period	Attack period	Scan volume	Size
fox	2022-01-15 to 2022-01-20	2022-01-18 11:59–13:15	High	26 GB
santos	2022-01-14 to 2022-01-18	2022-01-17 11:15–11:59	Low	17 GB

Table 8: Flow-label distribution for the DARPA 2000 LLDOS 1.0 scenario.

Sensor	Benign	Phase_1	Phase_2	Phase_3	Phase_4	Phase_5
Inside	52,258	20	22	34	19	73,472
DMZ	10,609	767	25	79	19	33,942

4. **Privilege Escalation:** a compromised account and an interactive reverse shell are used to obtain root privileges.
5. **Remote Command Execution:** the attacker executes commands, including operations requiring root privileges.
6. **Data Exfiltration:** DNSteal is used to exfiltrate data from an intranet file share.

Although the eight scenarios implement the same attack, they vary in scan order, duration, event frequency, and phase coverage. We use the *fox* and *santos* scenarios, which respectively exhibit high and low scan volumes. This contrast enables evaluation under different traffic and attack-frequency conditions. Table 7 summarizes the selected scenarios.

To the best of our knowledge, this is the first evaluation of multi-step attack detection on AIT-LDSv2.

Network-Observable Attack Sequence. AIT-LDSv2 was designed primarily for log-based analysis. Consequently, not every labeled attack phase is observable from network traffic alone. In particular, privilege escalation and remote command execution rely predominantly on host-level evidence. Because our evaluation uses only network traffic, these phases are excluded from the modeled attack sequence. Data exfiltration also differs from the expected logical progression. The dataset authors start DNSteal near the beginning of the simulation to contaminate the nominal training period and challenge anomaly-detection methods that assume attack-free training data [22]. Exfiltration therefore precedes the remaining attack phases in the recorded timeline, rather than representing the attacker's final objective. We preserve the dataset's observed temporal order and evaluate the following network-visible sequence: *(i)* **Data Exfiltration**, *(ii)* **Scanning**, *(iii)* **Exploitation**, and *(iv)* **Password Cracking**.

C Data Preparation

The data preparation pipeline comprises four stages: flow extraction, attack-phase labeling, neural input construction, and DEEPCHRONOS query generation.

C.1 Flow Extraction and Labeling

Raw PCAP traffic is converted into flow-level records using the Zeek network security monitor [45]. Zeek produces several protocol-specific logs. However, because DEEPCHRONOS assumes that packet payloads may be encrypted, we use only `conn.log`, which contains connection metadata derived from packet headers.

DARPA 2000. The dataset provides a complete traffic capture containing both benign and malicious activity, together with separate PCAP subsets for the five attack phases. Zeek is applied uniformly to all captures, and flows are matched using hashes computed over their connection attributes, excluding the unique Zeek identifier. A flow in the complete capture receives the label of its matching phase-specific flow; unmatched flows are labeled as benign. Table 8 reports the resulting distributions for the inside and DMZ sensors.

AIT-LDSv2. The PCAP files collected from different sensors are processed separately with Zeek and merged according to connection start time. We retain traffic within the official simulation period and remove duplicate flows reported by the same sensor with identical attributes [23, 22]. AIT-LDSv2 provides labels for flows extracted using Tstat. Because Tstat and Zeek produce different flow representations, labels cannot be transferred through exact record matching. We instead align the records using their source and destination addresses, source and destination ports, and temporal proximity. The original fine-grained labels are subsequently mapped to the attack phases modeled in this work using the accompanying labeling notebook. Table 9 shows the resulting distributions.

Table 9: Flow-label distribution for the selected AIT-LDS V2 scenarios.

Scenario	Benign	Phase_1	Phase_2	Phase_3	Phase_4
fox	400,024	14,678	56,443	28	61
santos	333,964	12,861	5,670	26	228

Table 10: Zeek `conn.log` attributes used by DEEPCHRONOS.

Attribute	Description	Usage
`ts`	Connection timestamp	Symbolic
`id.orig_h`	Source address	Symbolic
`id.resp_h`	Destination address	Symbolic
`id.orig_p`	Source port	Symbolic
`id.resp_p`	Destination port	Symbolic
`local_orig`	Internal source indicator	Symbolic + Neural
`local_resp`	Internal destination indicator	Symbolic + Neural
`proto`	Transport protocol	Symbolic + Neural
`service`	Identified application service	Neural
`duration`	Connection duration	Neural
`orig_bytes`	Originator payload bytes	Neural
`resp_bytes`	Responder payload bytes	Neural
`conn_state`	Connection state	Neural
`missed_bytes`	Bytes missed during capture	Neural
`orig_pkts`	Originator packet count	Neural
`orig_ip_bytes`	Originator IP bytes	Neural
`resp_pkts`	Responder packet count	Neural
`resp_ip_bytes`	Responder IP bytes	Neural
`ip_proto`	IP protocol number	Neural
`uid`	Unique connection identifier	Excluded
`history`	Packet-history flags	Excluded
`tunnel_parents`	Parent tunnel identifiers	Excluded

C.2 Neural Input Construction

Neural inputs combine connection-level metadata from `conn.log` with statistical features computed from recent network activity. Table 10 summarizes the selected Zeek attributes and their use in DEEPCHRONOS. The unique connection identifier `uid` is discarded, `history` is excluded to avoid a high-dimensional categorical encoding, and `tunnel_parents` is omitted because it is absent from the evaluated datasets.

Statistical Features. For every flow, we summarize the preceding 60 seconds of traffic. Source-oriented features comprise the number of initiated connections, unique destination addresses, and unique destination ports. Destination-oriented features comprise the number of incoming connections and unique source addresses. We additionally compute the proportions of connections exhibiting SYN failures, rejections, and resets. The 60-second aggregation interval was selected experimentally and is configurable: longer intervals capture slower behaviors but may also introduce unrelated traffic.

Feature Processing. Categorical features are one-hot encoded, whereas numerical features are scaled to the interval $[0, 1]$ using MinMax normalization. Table 11 lists the resulting neural feature groups.

Temporal Windows. The transformed flows are ordered chronologically and grouped into fixed-length sequences. Each sequence contains the current flow and its preceding flows and is used to predict the phase associated with the current flow. Larger windows expose the neural model to longer-term behavior but increase noise, computation, and inference latency. We therefore evaluate windows of 10 and 100 flows.

Inference Query Generation. Each temporal window is converted into a DeepProbLog query containing two complementary inputs. The preprocessed feature vectors form the input to the neural attack-phase classifier, while the current flow's protocol, endpoint location, and service-related attributes are supplied to the symbolic component. The query also identifies the flow under evaluation and requests its most likely attack phase. During inference, Deep-

Table 11: Features provided to the neural component.

Type	Features
Categorical	`local_orig`, `local_resp`, `proto`, `service`, `conn_state`, and `ip_proto`.
Numerical	`duration`, `orig_bytes`, `resp_bytes`, `missed_bytes`, `orig_pkts`, `orig_ip_bytes`, `resp_pkts`, `resp_ip_bytes`, `connections_per_src`, `unique_targets`, `unique_dports`, `connections_per_dst`, `unique_sources_per_dst`, `syn_failure_ratio`, `reject_ratio`, and `reset_ratio`.

Table 12: Experimental configurations evaluated in neural baseline models.

Configuration	Evaluated options
Feature sets	Base features, flow-only features
Window sizes	10, 100
Learning rates	10^{-3}, 10^{-4}

ProbLog combines the neural phase probabilities with the applicable attack-phase rules and the recorded attack history to produce the final phase distribution.

D Neural Ablation Study

We conduct an ablation study to select the neural configuration used in the subsequent experiments. For the multiclass and ensemble LSTM models, we vary three factors: the input feature set, temporal window size w, and learning rate lr. The *flow-only* feature set contains the selected Zeek connection attributes, whereas the *base* feature set additionally includes the statistical features computed over recent network activity. The complete set of evaluated configurations is reported in Table 12.

Each configuration is evaluated using five-fold cross-validation with early stopping. To mitigate the substantial class imbalance in both datasets, training batches are constructed using a weighted sampler. We report macro F1-score, accuracy, detection rate (DR), and false alarm rate (FAR), and rank configurations by macro F1-score.

AIT-LDSv2. Table 13 reports the eight highest-ranked configurations on AIT-LDSv2. The ensemble LSTM using the base feature set, $w = 100$, and $lr = 10^{-3}$ achieves the best result across all metrics, with a macro F1-score of 0.8650. The corresponding multiclass model ranks second. More generally, the strongest configurations use the base feature set and the longer temporal window, indicating that statistical features and extended flow history are beneficial for this dataset.

DARPA 2000. Table 14 presents the corresponding results for DARPA 2000. The ensemble LSTM using the base feature set, $w = 10$, and $lr = 10^{-3}$ obtains the highest macro F1-score and accuracy. The multiclass model with the same configuration ranks second and achieves the lowest FAR, while the flow-only ensemble obtains the highest DR. In contrast to AIT-LDSv2, the best DARPA 2000 configurations predominantly use the shorter temporal window, suggesting that the relevant attack behavior is captured within a more compact flow history.

Selected Configurations. Based on macro F1-score, we select the base feature set and a learning rate of 10^{-3} for both datasets. A window of 100 flows is used for AIT-LDSv2, whereas a window of 10 flows is used for DARPA 2000. These dataset-specific settings are applied consistently to the neural baselines and to the corresponding neural components of the DEEPCHRONOS models.

Table 13: Top neural configurations on AIT-LDS V2, ranked by macro F1-score. The best value for each metric is shown in bold.

Model	Features	Window	Learning rate	Macro F1	Accuracy	DR	FAR
$\text{LSTM}_{\text{Ensemble}}$	Base	100	10^{-3}	**0.8650**	**0.9966**	**0.9874**	**0.0029**
$\text{LSTM}_{\text{Multiclass}}$	Base	100	10^{-3}	0.8213	0.9952	0.9865	0.0043
$\text{LSTM}_{\text{Ensemble}}$	Base	100	10^{-4}	0.7498	0.9907	0.9867	0.0090
$\text{LSTM}_{\text{Multiclass}}$	Base	100	10^{-4}	0.6919	0.9874	0.9846	0.0124
$\text{LSTM}_{\text{Multiclass}}$	Base	10	10^{-3}	0.6749	0.9923	0.9756	0.0067
$\text{LSTM}_{\text{Ensemble}}$	Base	10	10^{-3}	0.6741	0.9924	0.9760	0.0066
$\text{LSTM}_{\text{Multiclass}}$	Flow-only	100	10^{-3}	0.6367	0.9307	0.9706	0.0715
$\text{LSTM}_{\text{Ensemble}}$	Base	10	10^{-4}	0.6365	0.9880	0.9742	0.0112

Table 14: Top neural configurations on DARPA 2000, ranked by macro F1-score. The best value for each metric is shown in bold.

Model	Features	Window	Learning rate	Macro F1	Accuracy	DR	FAR
$\text{LSTM}_{\text{Ensemble}}$	Base	10	10^{-3}	**0.9152**	**0.99958**	0.99957	0.00030
$\text{LSTM}_{\text{Multiclass}}$	Base	10	10^{-3}	0.9142	0.99957	0.99953	**0.00027**
$\text{LSTM}_{\text{Ensemble}}$	Flow-only	10	10^{-3}	0.9131	0.99955	**0.99959**	0.00041
$\text{LSTM}_{\text{Multiclass}}$	Flow-only	10	10^{-3}	0.8850	0.99949	0.99955	0.00046
$\text{LSTM}_{\text{Ensemble}}$	Base	100	10^{-3}	0.8843	0.99955	0.99953	0.00033
$\text{LSTM}_{\text{Ensemble}}$	Flow-only	100	10^{-3}	0.8817	0.99953	0.99951	0.00030
$\text{LSTM}_{\text{Multiclass}}$	Flow-only	100	10^{-3}	0.8793	0.99951	0.99953	0.00033
$\text{LSTM}_{\text{Ensemble}}$	Flow-only	10	10^{-4}	0.8783	0.99944	0.99953	0.00055